

Séance 02 - Premiers programmes PHP avec Docker

Table of Contents

1. Situation professionnelle	1
2. Objectifs	2
3. Prérequis	2
4. Réactivation - Git	2
5. Activité 1 - Préparer l'espace de travail	3
6. Activité 2 - Démarrer le serveur PHP	3
7. Activité 3 - Créer le premier programme PHP	3
7.1. Créer le fichier	3
7.2. Exécuter le fichier	4
7.3. Comprendre le trajet du fichier	4
7.4. Provoquer puis corriger une erreur	4
7.5. Premier commit de la séance	4
8. Activité 4 - Variables et types simples	5
8.1. Déclarer des variables	5
8.2. Règles de nommage	5
8.3. Examiner les types	6
9. Activité 5 - Calculs et opérateurs	6
9.1. Calculer le coût d'un parc	6
9.2. Comprendre la priorité des opérateurs	7
10. Activité 6 - Diagnostic d'un programme	7
11. Exercice autonome - Présenter un poste informatique	8
12. Travail sur papier	8
13. Exercices bonus	9
13.1. Exercice bonus 1 [*] - Répartir des équipements en baies	9
13.2. Exercice bonus 2 [**] - Quel est le type du résultat ?	10
13.3. Exercice bonus 3 [*] - Défi : prévoir la sortie	10
14. Synthèse	10
15. Validation Git	11
16. Arrêter le serveur PHP	11

1. Situation professionnelle

Vous intégrez une équipe chargée de développer progressivement un outil de gestion et de supervision d'un parc informatique.

Avant de manipuler des équipements, des utilisateurs ou une base de données, vous devez maîtriser les éléments les plus simples d'un programme : afficher un résultat, stocker des valeurs, reconnaître leur type et effectuer des calculs.

Le code PHP sera exécuté dans un conteneur Docker identique sur tous les postes.

2. Objectifs

À la fin de la séance, vous devrez être capable de :

- démarrer et vérifier le conteneur PHP ;
- expliquer le lien entre un fichier Windows, le conteneur et son URL ;
- créer et exécuter un fichier PHP ;
- déclarer des variables avec des noms explicites ;
- distinguer les types `string`, `int`, `float` et `bool` ;
- afficher une valeur avec `echo` ;
- examiner une variable avec `var_dump()` ;
- utiliser les principaux opérateurs arithmétiques ;
- concaténer du texte et des variables ;
- enregistrer le travail dans Git.

3. Prérequis

- La séance 01 sur Git est terminée.
- Docker Desktop est installé sur le poste et **démarré** (icône baleine en bas à droite).
- Vous avez le dossier du TP donné par le professeur. Il contient déjà `compose.yml`, une page `index.php` et `diagnostic.php`.
- Visual Studio Code est disponible.



Le guide d'installation pas à pas est dans `cours/annexe-docker-php.adoc`. En résumé : ouvrez le dossier du TP, tapez `docker compose up -d`, puis ouvrez <http://localhost:8181>.

4. Réactivation - Git

Sans utiliser vos notes, répondez sur papier.

1. Quelle commande affiche l'état du dépôt ?
2. Quelle commande récupère les derniers commits distants ?
3. Quelle commande prépare un fichier pour le prochain commit ?

4. Quelle commande enregistre une version dans le dépôt local ?
5. Quelle commande publie les commits locaux ?

5. Activité 1 - Préparer l'espace de travail

Tout se passe dans **un seul dossier** : celui du TP, qui contient déjà `compose.yml`. Vos fichiers `.php` iront **dans ce dossier**, et vous les ouvrirez à l'adresse <http://localhost:8181/>.

Élément	À retenir
Le dossier de travail	celui du TP (il contient <code>compose.yml</code>) — vos fichiers <code>.php</code> vont ici
L'adresse du serveur	http://localhost:8181



Un fichier `.php` placé **ailleurs** que dans le dossier du TP ne s'affichera pas.

Ouvrez le dossier du TP dans Visual Studio Code : *Fichier* → *Ouvrir le dossier...* Le fichier `diagnostic.php` est déjà dans le dossier (il servira à l'activité 6).

6. Activité 2 - Démarrer le serveur PHP

Docker Desktop doit être démarré (icône baleine en bas à droite).

Dans VS Code, ouvrez un terminal (*Terminal* → *Nouveau terminal*) et tapez :

```
docker compose up -d
```

La première fois, patientez (ça télécharge PHP). Puis ouvrez le navigateur à l'adresse :

```
http://localhost:8181/
```

Une page verte « **Bravo, PHP fonctionne !** » doit s'afficher, avec la version de PHP. Recopiez cette version dans votre compte rendu.

QUESTION: PHP est-il exécuté directement par Windows ou à l'intérieur du conteneur Docker ?

7. Activité 3 - Créer le premier programme PHP

7.1. Créer le fichier

Dans le dossier du TP, créez `bonjour.php` :

```
<?php  
  
echo "Bonjour, PHP fonctionne dans Docker !";
```



Dans un fichier contenant uniquement du PHP, la balise fermante `?>` est inutile et sera omise.

7.2. Exécuter le fichier

Dans le navigateur, ouvrez l'adresse suivante :

```
http://localhost:8181/bonjour.php
```

Vous devez obtenir :

```
Bonjour, PHP fonctionne dans Docker !
```

7.3. Comprendre le trajet du fichier

Complétez le tableau.

Élément	Valeur
Chemin Windows du fichier	...
Chemin du fichier dans le conteneur	...
URL utilisée par le navigateur	...

7.4. Provoquer puis corriger une erreur

Supprimez temporairement le point-virgule à la fin de l'instruction `echo`, enregistrez puis rechargez la page.

1. Quel type d'erreur est indiqué ?
2. Quel numéro de ligne est signalé ?
3. Pourquoi l'erreur apparaît-elle seulement après le rechargement du navigateur ?

Rétablissez ensuite le point-virgule.

7.5. Premier commit de la séance

Depuis la racine de votre projet :

```
git status
git add bonjour.php
git commit -m "Ajoute le premier programme PHP"
git push
```

8. Activité 4 - Variables et types simples

8.1. Déclarer des variables

Créez `equipement.php` :

```
<?php

$nomEquipement = "srv-web-01";
$memoireGo = 16;
$stockageGo = 512;
$coutMensuel = 24.90;
$estActif = true;

echo "Nom : " . $nomEquipement . "<br>";
echo "Mémoire : " . $memoireGo . " Go<br>";
echo "Stockage : " . $stockageGo . " Go<br>";
echo "Coût mensuel : " . $coutMensuel . " euros<br>";
```

Exécutez ce programme dans le navigateur.



Pas de Docker sous la main ? Une version autonome équivalente est disponible dans [cours/sandbox-online/02-variables.php](https://3v4l.org) : copiez-collez-la sur <https://3v4l.org> ou <https://onlinephp.io> pour observer `echo` et `var_dump()` sans serveur.

8.2. Règles de nommage

Un nom de variable PHP :

- commence par le caractère `$` ;
- commence ensuite par une lettre ou `_` ;
- peut contenir des lettres, des chiffres et `_` ;
- ne contient ni espace ni tiret ;
- doit décrire la donnée stockée.

Par convention, nous utiliserons le `camelCase`.

Indiquez si les noms suivants sont valides et justifiez les réponses incorrectes.

Nom proposé	Valide ?	Justification
\$nomServeur	...	...
\$2serveurs	...	...
\$nombre_serveurs	...	...
\$adresse-ip	...	...
nomServeur	...	...
\$_adresseIp	...	...

8.3. Examiner les types

Ajoutez à la fin de `equipement.php` :

```
echo "<hr>";

var_dump($nomEquipement);
var_dump($memoireGo);
var_dump($coutMensuel);
var_dump($estActif);
```

Associez chaque variable à son type : `string`, `int`, `float` ou `bool`.

QUESTION: Pourquoi `echo $estActif;` est-il moins utile que `var_dump($estActif);` pour observer un booléen ?

9. Activité 5 - Calculs et opérateurs

Opérateur	Rôle	Exemple
+	Addition	<code>\$a + \$b</code>
-	Soustraction	<code>\$a - \$b</code>
*	Multiplication	<code>\$a * \$b</code>
/	Division	<code>\$a / \$b</code>
%	Reste d'une division entière	<code>\$a % \$b</code>
.	Concaténation	<code>"Nom : " . \$nom</code>

9.1. Calculer le coût d'un parc

Créez `cout-parc.php` avec :

```
<?php
```

```
$nombreServeurs = 6;  
$coutMensuelUnitaire = 24.90;  
$nombreMois = 12;
```

Complétez le programme pour calculer et afficher :

- le coût mensuel de tous les serveurs ;
- le coût annuel du parc ;
- le coût moyen mensuel par serveur.

Résultat attendu :

```
Nombre de serveurs : 6  
Coût mensuel du parc : 149.4 euros  
Coût annuel du parc : 1792.8 euros  
Coût moyen par serveur : 24.9 euros
```

CONTRAINTE: Les résultats doivent être obtenus par des calculs et non écrits directement dans les **echo**.

9.2. Comprendre la priorité des opérateurs

Sans exécuter le code, prévoyez les valeurs :

```
$nombreServeurs = 6;  
$cout = 10;  
$fraisFixes = 20;  
  
$resultat1 = $nombreServeurs * $cout + $fraisFixes;  
$resultat2 = $nombreServeurs * ($cout + $fraisFixes);
```

Vérifiez ensuite dans **priorites.php**.

QUESTION: Quel est le rôle des parenthèses dans la deuxième expression ?

10. Activité 6 - Diagnostic d'un programme

Ouvrez **diagnostic.php** (déjà dans le dossier du TP). Il contient plusieurs erreurs.

1. Relevez chaque erreur.
2. Expliquez sa cause.
3. Corrigez le fichier.
4. Exécutez-le dans le navigateur : <http://localhost:8181/diagnostic.php>.

Le programme corrigé doit afficher une fiche d'équipement et son coût annuel.

Après correction :

```
git diff -- diagnostic.php
```

11. Exercice autonome - Présenter un poste informatique

Créez `poste.php` avec au minimum :

- le nom du poste ;
- le système d'exploitation ;
- la mémoire vive ;
- la capacité du disque ;
- le prix d'achat ;
- la durée prévue d'utilisation en années ;
- l'état d'utilisation sous forme de booléen.

Calculez le coût moyen annuel puis affichez toutes les informations.

CONTRAINTES:

- noms de variables explicites ;
- au moins un calcul avec des parenthèses ;
- chaînes construites à partir des variables ;
- `var_dump()` utilisé pour vérifier le booléen ;
- aucune condition dans cette séance.

12. Travail sur papier

Sans ordinateur, étudiez :

```
<?php

$nom = "pc-sio-12";
$prix = 840.0;
$ duree = 4;
$coutAnnuel = $prix / $duree;
$description = $nom . " : " . $coutAnnuel . " euros par an";
```



```
echo $description;
```

1. Donnez le type des cinq variables.
2. Donnez la valeur de `$coutAnnuel`.
3. Écrivez exactement le texte affiché.
4. Expliquez le rôle de l'opérateur `..`
5. Réécrivez le calcul pour obtenir le coût mensuel moyen.

13. Exercices bonus

Facultatifs et gradués, dans le périmètre de la séance (variables, types, opérateurs, concaténation, `var_dump()` ; ni condition, ni boucle, ni tableau). Ils sont autonomes : testez-les dans Docker ou en ligne sur <https://3v4l.org>.

- `[*]` accessible ;
- `[**]` intermédiaire ;
- `[*]` défi.

13.1. Exercice bonus 1 `[*]` - Répartir des équipements en baies

On installe des équipements dans des baies (racks) de capacité fixe. Créez `baies.php` :

```
<?php  
  
$nombreEquipements = 29;  
$capaciteBaie = 8;
```

La fonction `intdiv($a, $b)` donne le quotient **entier** de la division ; l'opérateur `%` donne le **reste**. Complétez le programme pour afficher :

- le nombre de baies complètement remplies ;
- le nombre d'équipements restant dans la dernière baie incomplète.

Sortie attendue :

```
Baies pleines : 3  
Équipements dans la dernière baie : 5
```

13.2. Exercice bonus 2 **[**]** - Quel est le type du résultat ?

En PHP, l'opérateur `/` produit toujours un nombre décimal. Vérifiez-le. Créez `types-calcul.php` :

```
<?php

$totalIncidents = 30;
$nombreTechniciens = 4;
$moyenne = $totalIncidents / $nombreTechniciens;

$memoireGo = 12;
$memoireDoublee = $memoireGo * 2;
```

1. Avant d'exécuter, prévoyez le **type** et la **valeur** de `$moyenne` et de `$memoireDoublee`.
2. Ajoutez `var_dump($moyenne);` et `var_dump($memoireDoublee);` et vérifiez.
3. Expliquez en une phrase pourquoi `$memoireDoublee` n'est pas du même type que `$moyenne`.

13.3. Exercice bonus 3 **[*]** - Défi : prévoir la sortie

PHP « jongle » avec les types selon l'opérateur utilisé. Sans exécuter, prévoyez la sortie **exacte** de chaque ligne (type et valeur), puis vérifiez dans un fichier `jonglage.php` :

```
<?php

var_dump(7 % 3);
var_dump(10 / 4);
var_dump("srv-" . 3);
var_dump(true + 1);
var_dump(5 . 5);
```

Pour chaque ligne, expliquez en quelques mots la règle appliquée (reste entier, division décimale, concaténation, conversion d'un booléen en nombre, concaténation de deux entiers).

14. Synthèse

1. Un fichier PHP possède l'extension ...
2. Une variable PHP commence par ...
3. L'instruction utilisée pour afficher une valeur est ...
4. La fonction qui affiche la valeur et le type est ...
5. Le type d'un entier est ...
6. Le type d'un nombre décimal est ...

7. L'opérateur de concaténation est ...
8. Le dossier de votre projet servi par le conteneur (où vous placez vos fichiers) est ...
9. L'adresse du serveur PHP local commence par ...

15. Validation Git

```
git status
git add .
git commit -m "Ajoute les exercices sur les variables et les opérateurs"
git push
git status
git log --oneline
```

Le dépôt doit être propre et les deux nouveaux commits visibles sur la forge.

16. Arrêter le serveur PHP

En fin de séance, dans le terminal (vos fichiers restent) :

```
docker compose down
```